

1. Google 云计算技术包括哪些内容？

Google 云计算技术主要包括分布式存储、分布式计算、分布式协调与管理以及应用平台等方面。在分布式存储方面，有 Google 文件系统 GFS、分布式结构化数据表 Bigtable 以及分布式存储系统 Megastore；在分布式数据处理方面，有 MapReduce 以及面向海量数据分析的 Dremel 和 PowerDrill；在分布式协调与系统管理方面，有分布式锁服务 Chubby 和大规模分布式系统监控基础架构 Dapper；此外，Google 还提供了应用程序托管平台 Google App Engine，用于支持云端应用的开发和运行。

2. 当前主流分布式文件系统有哪些？各有什么优缺点？

当前主流的分布式文件系统主要包括 Google 的 GFS、HDFS、RedHat 的 GFS、IBM 的 GPFS、Sun 的 Lustre 以及阿里巴巴的 TFS 等。Google File System (GFS) 是 Google 为内部大规模数据处理设计的分布式文件系统，适合大文件顺序读写，采用大块存储和多副本机制，具有高吞吐量和良好的容错性，但主要用于 Google 内部，通用性较差。HDFS 是 Hadoop 生态中的分布式文件系统，其设计思想与 GFS 类似，适合批处理和大数据分析，具有良好的扩展性和容错性，但不适合低延迟访问和大量小文件存储。Global File System (RedHat GFS) 是一种面向集群环境的共享磁盘文件系统，支持多节点并发访问，具有良好的一致性，但对底层存储和硬件依赖较强。GPFS (IBM) 和 Lustre (Sun) 多用于高性能计算环境，具有高带宽和并行 I/O 能力，适合科学计算和大规模并行应用，但系统复杂、部署成本较高。TFS (Taobao File System) 主要面向海量小文件存储，支持平滑扩容和高并发访问，适合互联网业务场景，但不适合超大文件的顺序读写。

3. GFS 采用了哪些容错措施来确保整个系统的可靠性？

GFS 在设计时假设系统中的节点和硬件会频繁发生故障，因此采用了多层次的容错机制来保证系统的可靠性。在系统架构上，GFS 采用中心服务器模式，由 Master 管理元数据，Chunk Server 负责数据存储，并通过控制信息与数据分离，避免 Master 成为系统性能和可靠性上的瓶颈。在 Master 容错方面，GFS 通过操作日志 (Operation Log) 记录所有元数据的更新操作，当 Master 发生故障时，在磁盘数据保持完好的情况下可以迅速恢复元数据；同时，GFS 还提供了 Master 的远程实时备份机制，以防止 Master 彻底失效。在 Chunk Server 容错方面，GFS 采用多副本机制，将数据块的副本存储在不同的 Chunk Server 上，当某个 Chunk Server 发生故障时，系统可以通过其他副本继续提供服务，并自动进行副本重建。通过上述容错措施，GFS 能够在使用廉价硬件的情况下保证系统的高可靠性和可用性。

4. MapReduce 与传统的分布式程序设计相比有何优点？

与传统的分布式程序设计相比，MapReduce 的主要优点在于对分布式系统的复杂性进行了高度封装。MapReduce 屏蔽了并行处理、任务调度、容错处理、数据本地化以及负载均衡等底层细节，为程序员提供了一个简单而统一的编程模型，使开发者只需关注 Map 和 Reduce 两个函数的实现。在运行过程中，MapReduce 将对大规模数据集的操作分解为多个子任务，由主节点统一调度，在各个工作节点上并行执行，当节点发生故障时，系统能够自动进行任务重试，从而保证任务的可靠执行和容错性。因此，MapReduce 大大降低了分布式程序开发的难度，提高了系统的可靠性和可扩展性。

5. Chubby 的设计目标是什么？Paxos 算法在 Chubby 中起什么作用？

Chubby 是 Google 设计的一种用于松耦合分布式系统的协调服务，主要提供粗粒度的建议性锁、命名和配置信息存储，用于解决分布式系统中的一致性问题。Chubby 的设计目标包括：以高可用性和高可靠性为首要目标，在此基础上再考虑系统性能；支持粗粒度的建议性锁服务，以降低锁管理的复杂度并提高系统整体性能；支持服务信息的直接存储，便于统一管理系统配置和元数据；同时提供通报机制和一致性缓存机制，使客户端能够及时感知状态变化并减少对服务器的频繁访问。Paxos 算法在 Chubby 中用于保证多个副本之间的一致性，通过 Paxos 进行 leader 选举和状态同步，确保在节点发生故障的情况

下，系统仍能对锁状态和服务信息（如锁状态、命名和配置信息）达成一致，从而保证 Chubby 的高可靠性和一致性。在实现上，Chubby 的底层采用了容错日志机制，所有对系统状态的修改都会记录到日志中，而日志条目的顺序和内容由 Paxos 算法在多个副本之间协商决定，从而保证不同副本上日志的一致性，进而保证系统整体的一致性和可靠性。

6. 阐述 Bigtable 的数据模型和系统架构。

Bigtable 是 Google 设计的一种建立在 Google WorkQueue, GFS 和 Chubby 之上的分布式存储系统，用于管理海量结构化数据。在数据模型方面，Bigtable 将数据组织为一个分布式的多维映射表，数据通过行关键字（Row Key）、列关键字（Column Key）和时间戳（Timestamp）进行索引。Bigtable 不对数据内容进行解析，所有数据都被视为字节串，具体的数据结构由应用程序自行定义，从而具有很强的灵活性。在系统架构方面，Bigtable 主要由客户端、一个 Master 服务器以及多个 Tablet Server 组成。Tablet 是数据分布和负载均衡的基本单位，负责存储表中的一部分行数据。Tablet Server 负责管理多个 Tablet，并处理客户端的读写请求。Master 服务器主要负责 Tablet 的分配、负载均衡、故障恢复以及表的元数据管理，而不参与实际的数据读写。Bigtable 依赖 GFS 存储底层数据，并通过 Chubby 提供的分布式锁和命名服务来完成 Tablet 定位、Master 选举和系统协调，从而保证系统的高可靠性和可扩展性。

7. 分布式存储系统 Megastore 的核心技术是什么？

Megastore 是构建在 Bigtable 之上的分布式存储系统，其核心技术是基于 Paxos 算法的同步复制机制，通过在多个副本之间达成一致来支持跨数据中心的强一致性事务。Megastore 通过复制日志来实现数据的一致性，每个副本都保存记录所有更新操作的日志，并通过 Paxos 算法保证不同副本上的日志内容和顺序一致。即使副本处于故障恢复过程中，也能够参与到写操作的 Paxos 共识中。在数据读取过程中，Megastore 通过追赶（Catchup）机制，确保在执行一次一致性读之前，至少有一个副本已经应用了所有已提交的日志，从而保证读取结果的正确性。在数据写入过程中，所有更新操作会被打包并作为 Paxos 提议提交，当该提议获得分布式共识后，更新将被应用到所有副本；否则事务中止并重新执行。为了保证协调者的可用性，Megastore 借助 Chubby 锁服务来管理协调者状态，通过多数锁机制防止错误写入，从而在协调者故障时保护系统的一致性。

8. 大规模分布式系统的监控基础架构 Dapper 的关键技术是什么？

Dapper 的设计目标包括低开销、对应用层透明、可扩展性，其中对应用层透明是实现难度最大的目标。为实现上述目标，Google 主要采用了以下两项关键技术：（1）轻量级核心功能库 Dapper 将监控的核心功能封装在一个轻量级的小规模核心功能库中，该库主要由通用线程、控制流和 RPC 代码库组成。Dapper 的核心监控机制正是在该小规模库之上实现的，其主要功能包括区间（Span）创建、请求抽样以及在本地磁盘上记录日志。在实现上，该核心功能库规模较小：C++ 实现代码少于 1000 行，Java 实现代码少于 800 行，键/值对方式的注释功能仅需额外增加约 500 行代码。通过将复杂功能限制在轻量级核心功能库中，Dapper 保证了监控过程基本不影响应用程序运行，实现了对应用层的透明性。（2）二次抽样技术由于 Google 每天需要处理的请求量巨大，若对所有请求进行监控，将导致监控开销过大且不利于数据分析。因此，Dapper 采用了二次抽样技术来降低监控成本。在第一次抽样中，Dapper 对请求进行抽样，只有被抽中的请求才会被监控。实践表明，即使将抽样率降低至 $1/1024$ ，即在 1024 个请求中抽取 1 个进行监控，仍能够获得足够多的有效监控数据。低抽样率仍然有效的原因在于：系统事件数量巨大，关键事件出现频率较高，从而可以被成功捕获。最初 Dapper 采用统一抽样率，但后来发现对流量较低的服务可能忽略关键事件，因此进一步引入了适应性抽样率方案。

9. 相比于行存储，列存储有哪些优点？

相比于行存储，列存储具有以下优点：1. 减少无关数据的读取，提高查询效率 列存储以属性（列）为单位进行存储，在数据处理过程中，只需要读取查询所涉及的列数据，而无需加载整行记录，从而减少

I/O 开销，提高查询效率，特别适合分析型查询场景。2. 更利于数据压缩，节省存储空间 同一列中的数据类型通常相同，数据分布特征相似，列存储更容易采用高效的压缩算法进行压缩，从而显著降低存储空间占用并提升缓存命中率。3. 适合松散、扁平或嵌套的数据结构 现实应用中，很多数据之间并不存在严格的关系，而是呈现松散和扁平的结构。列存储结合嵌套数据模型，能够更灵活地组织和存储此类数据，适合 Google 等大规模数据存储与分析需求。4. 有利于顺序存储和批量处理 在关系型数据库中，不同列中相同位置的数据必然属于同一行，因此可以直接将每一列的值按顺序存储，不会丢失数据信息，便于进行大规模顺序扫描和批量处理。

10. 为什么 MapReduce 不适合实时数据处理？

MapReduce 作为一种面向批处理的数据处理框架，虽然具有实现简单、易于扩展等优点，但并不适合实时数据处理，主要原因如下：1. 处理延迟高，无法满足实时性要求 MapReduce 面向的是海量数据的离线批处理任务。由于数据规模巨大，一个 MapReduce 作业通常需要运行数小时甚至更长时间，整体处理延迟较高，无法在用户可接受的时间内返回结果。2. 不适合频繁的交互式数据分析 在数据探索 (Data Exploration) 场景中，数据分析人员往往希望在编写或修改代码后，能够立刻验证结果。如果使用 MapReduce，每次修改算法后都需要重新提交作业并长时间等待结果，整个过程可能反复多次，总耗时甚至达到数天，严重影响分析效率。3. 缺乏实时交互能力 实际应用中，用户更期望像传统 SQL 查询那样进行实时的数据交互，即在提交请求后，系统能够在一个相对合理的时间内返回结果。而 MapReduce 无法支持这种低延迟、交互式的查询需求。4. 不适应实时数据交互需求的增长趋势 随着对实时数据交互应用需求的不断增强，MapReduce 的高延迟特性逐渐成为瓶颈。为满足实时交互式查询需求，Google 在借鉴搜索引擎和并行数据库技术的基础上，进一步开发了实时交互式查询系统 Dremel。

11. 简要阐述 Dremel 如何实现数据的无损压缩。

Dremel 通过对嵌套数据结构进行无损表示，在列存储的基础上实现了数据的无损压缩，其核心思想是在存储列值的同时，记录必要的结构信息。具体做法如下：1. 引入重复深度 (repetition level, r) 和定义深度 (definition level, d) 为准确表示嵌套数据结构，Dremel 为每个列值定义了两个变量：- 重复深度 r：表示该列值对应的“可重复 (repeated) 字段”是在哪一层级发生重复的，用于刻画嵌套结构中的重复关系；- 定义深度 d：表示该值路径中所有可能未定义 (非 required, 包括 repeated 和 optional) 的字段中，实际被定义的字段数量，用于刻画字段是否缺失。2. 仅针对必要字段记录结构信息，避免冗余存储 重复深度只关注 repeated 类型字段，不考虑 required 和 optional 字段；定义深度同时关注 repeated 和 optional 类型字段。通过只记录必要的结构信息，既能完整表达嵌套关系，又减少了冗余存储。3. 列数据以块 (Block) 形式存储，实现高效压缩 每一列最终被存储为多个块 (Block) 的集合，每个块中包含：- 字段值 - 对应的重复深度 r - 对应的定义深度 d 4. 利用定义深度隐式表示 NULL 值 NULL 值无需显式存储。当某个列值的定义深度小于其路径中可重复和可选字段数量之和时，即可判定该值为 NULL，从而减少存储开销。5. 对必需字段和冗余信息进行省略优化 对于 required 字段，其值不需要定义深度；在某些情况下，若定义深度 d 已经确定重复深度 r (如 $d = 0$ 时 $r = 0$)，则重复深度可以省略，进一步提高压缩效率。

12. PowerDrill 能够实现高效的数据处理，在存储部分主要依赖哪两方面的技术？

PowerDrill 在存储部分实现高效数据处理，主要依赖以下两方面的技术：(1) 基于列式存储和数据分块的裁剪技术 在对内部数据和查询类型分析后，PowerDrill 基于“查询模式高度相似、热点数据占比小”的假设，对数据采用列式存储并进行分块管理。通过为每个数据块设计巧妙的数据结构，PowerDrill 能够在查询阶段判断哪些数据块与查询条件无关，并将其直接略去，从而大幅减少查询过程中需要读取和处理的数据量。这种方式相比传统关系数据库依赖索引的方法，更适合 ad hoc 查询场景，在索引难以发挥作用时仍能保持高效查询性能。(2) 数据编码与压缩优化 (基于基数估计) 为减少数据在内存中的占用、提高内存利用率，PowerDrill 对块内数据进行高效编码与压缩。具体而言，系统会根据每个数

据块中不同值的数量（即数据的基数），选择合适的可变比特位数对数据进行编码，从而降低存储开销。在大规模数据环境下，PowerDrill 并不进行精确的基数统计，而是采用基数估计方法。Google 内部使用的是 HyperLogLog 的一种变种算法，在保证一定精度的前提下高效完成基数估计，为数据编码优化提供依据。总结：PowerDrill 在存储层主要通过 列式存储下的数据分块裁剪 以及 基于基数估计的数据编码与压缩优化 两方面技术，实现了高效的数据处理和内存计算能力。

13. Google App Engine 提供了哪些服务？

Google App Engine (GAE) 为应用程序提供了完整的运行环境和多种内置服务，主要包括以下几类：1. 图像操作服务 (Images API) 提供图像处理功能，支持图像的加载、缩放、裁剪等操作，主要包括 Image 类 及相关的 exception 类。2. 邮件服务 (Mail API) 支持应用程序发送电子邮件，规定了允许的附件类型，并通过 EmailMessage 类 构造和发送邮件。3. 缓存服务 (Memcache API) 提供基于内存的高速缓存机制，支持通过构造函数和实例方法对缓存数据进行读写，以减少对数据库的访问、提高系统性能。4. 用户管理服务 (User API) 提供用户身份认证与管理功能，支持 User 对象、User 类 以及用户登录网址，方便应用使用 Google 账户进行用户登录和身份验证。5. 数据库服务 (Data Store API) 提供持久化的数据存储功能，支持结构化数据的存储与查询，主要包括 Model 类、Property 类、Query 类、GqlQuery 类和 Key 类，用于数据建模、查询和管理。总结：Google App Engine 通过提供图像处理、邮件、缓存、用户管理以及数据库等一系列服务，屏蔽了底层基础设施的复杂性，使开发者能够专注于应用程序逻辑的开发。

14. Google App Engine 的沙盒对开发人员有哪些限制？

Google App Engine 通过受限沙盒机制对应用程序进行环境隔离，为开发人员带来了以下主要限制：1. 对外部网络访问受限 应用程序只能通过 Google App Engine 提供的 网址抓取 API 和 电子邮件服务 API 访问互联网中的其他计算机；外部计算机若要与该应用程序通信，只能通过 HTTP 或 HTTPS 等标准接口进行访问。2. 文件系统访问受限 应用程序无法对 Google App Engine 的文件系统进行写入操作，只能读取应用程序代码中自带的文件；应用程序运行期间需要持久保存的数据，必须使用 Google App Engine 提供的 Data Store 数据库进行存储。3. 程序执行方式和运行时间受限 应用程序只能在响应网络请求时运行，且每次请求的处理时间必须非常短，通常需要在几秒之内完成；同时，请求处理程序在发送响应之后，不能再产生子进程或继续执行代码。